Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security



Memory Safety Vulnerabilities

Memory Safety



"Consider the example of an application that maintains to do lists for many users.

If I have a to do list with ten items, and I ask for the eleventh item, what should happen? […] In a **memory unsafe language**, unless the programmer explicitly checks which item they're asking for against the length of the list, the item that happens to be at that position in memory is fetched […]. Allowing programs to read past the front or end of a list is called an **out-of-bounds read**. […] if we simply allow someone to ask for the eleventh element of a ten item list, they may get to read the first item out of someone else's list! […]

A closely related spatial vulnerability is an **out-of-bounds write**. In this case imagine we tried to change the eleventh or negative first item in our to do list. Now we are changing someone else's to do list."

https://alexgaynor.net/2019/aug/12/introduction-to-memory-unsafety-for-vps-of-engineering/

Buffer Overflow



Out-of-bounds write is commonly known as buffer overflow.

NIST's definition:

> "A condition at an interface under which more input can be placed into a buffer or data holding area than the capacity allocated, overwriting other information. Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system."

https://csrc.nist.gov/glossary/term/buffer_overflow

Example

```
int main(int argc, char *argv[]) {
    int valid = FALSE;
    char str1[8];
    char str2[8];

    next_tag(str1);
    gets(str2);
    if (strncmp(str1, str2, 8) == 0)
       valid = TRUE;
    printf("buffer1: str1(%s), str2(%s), valid(%d)\n", str1, str2, valid);
}
```

Load a password e.g., "START" from a file/db into str1

(a) Basic buffer overflow C code

```
$ cc -g -o buffer1 buffer1.c
$ ./buffer1
START
buffer1: str1(START), str2(START), valid(1)
$ ./buffer1
EVILINPUTVALUE
buffer1: str1(TVALUE), str2(EVILINPUTVALUE), valid(0)
$ ./buffer1
BADINPUTBADINPUT
buffer1: str1(BADINPUT), str2(BADINPUTBADINPUT), valid(1)
```

Typical input

Unexpectedly long input

Malicious input



(b) Basic buffer overflow example runs

State of Stack

higher addresses

lower addresses

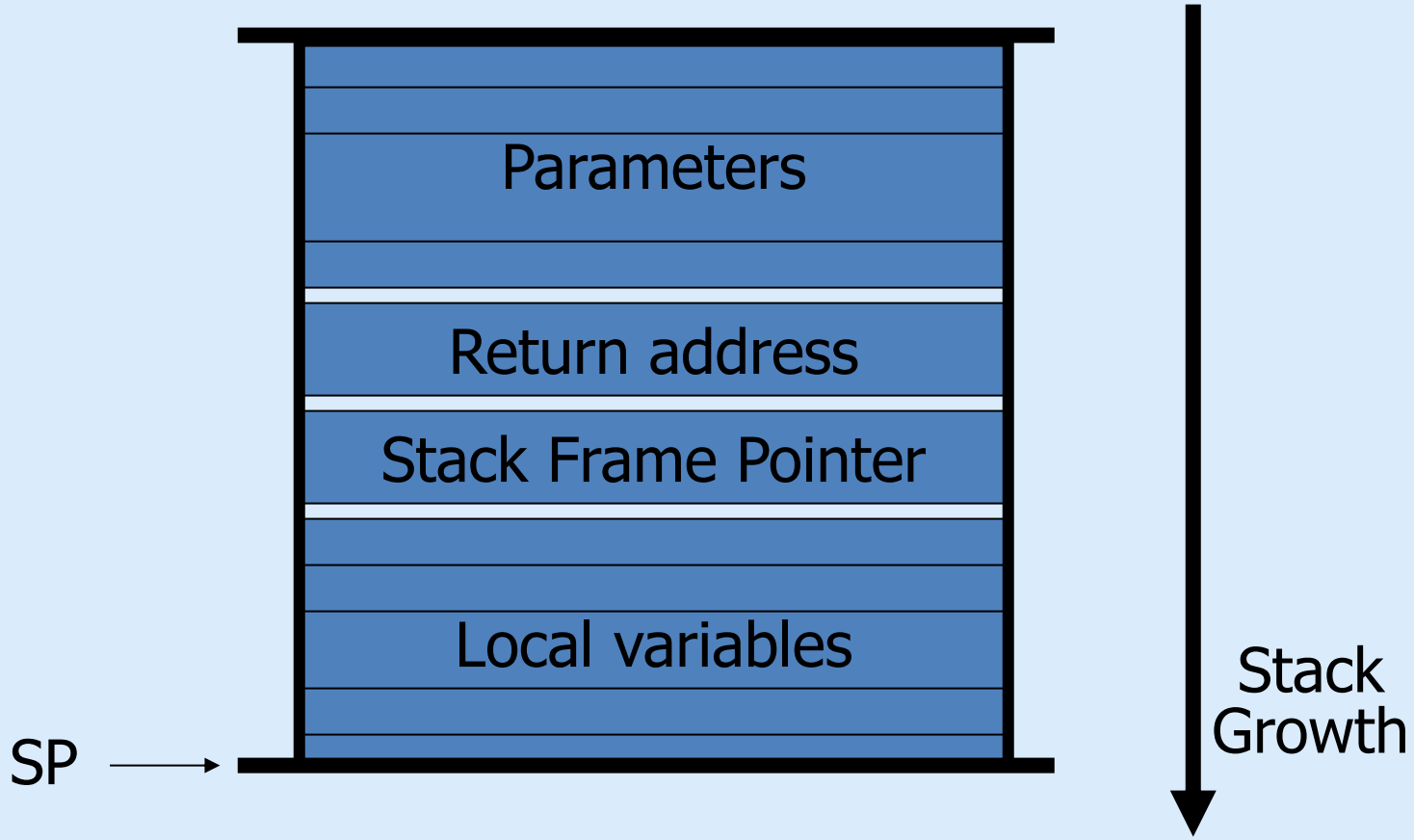
Memory Address	Before gets(str2)	After gets(str2)	Contains value of
. . . .	. . . .	. . . .	
bffffbf4	34fcffbf 4 . . .	34fcffbf 3 . . .	argv
bffffbf0	01000000	01000000	argc
bffffbec	c6bd0340 . . . @	c6bd0340 . . . @	return addr
bffffbe8	08fcffbf	08fcffbf	old base ptr
bffffbe4	00000000	01000000	valid
bffffbe0	80640140 . d . @	00640140 . d . @	
bffffbdc	54001540 T . . @	4e505554 N P U T	str1[4-7]
bffffbd8	53544152 S T A R	42414449 B A D I	str1[0-3]
bffffbd4	00850408	4e505554 N P U T	str2[4-7]
bffffbd0	30561540 0 V . @	42414449 B A D I	str2[0-3]
. . . .	. . . .	. . . .	



Remember: In general, stack grows backwards (decreasing addresses)

Stack structure during function call





Buffer Overflow: Basics



- Caused by programming error
- Allows more data to be stored than capacity available in a fixed sized buffer
  - buffer can be on stack, heap, global data
- Overwriting adjacent memory locations can cause:
  - corruption of program data
  - bypassing the security checks
  - unexpected transfer of control
  - memory access violation
  - execution of code chosen by attacker (**control hijacking**)

Buffer Overflow Attacks



- To exploit a buffer overflow an attacker must identify a buffer overflow vulnerability in some program.
  - they can do memory inspection, trace execution, or use fuzzing tools (feed the program with lots and lots of different inputs)

- Next, they need to understand how buffer is stored in memory and determine potential for corruption

Programming languages history



- At machine level, all data is an array of bytes
  - interpretation depends on instructions used
- Modern high-level languages (java, c#, python etc.) have a strong notion of data type, valid operations and most importantly buffer bounds checking
  - not vulnerable to buffer overflows
  - these extra checks incur a run-time overhead
- C and related languages have high-level control structures, but allow direct access to memory. There is no notion of array (buffer) bounds **at run time**
  - hence are vulnerable to buffer overflow
  - we have a large legacy of widely used, unsafe, and hence vulnerable C/C++ code

Why use lower level languages



- Systems software has historically been written in lower level languages (C, C++, assembly). These include:
  - operating systems, device drivers, file systems
  - network servers, database servers
  - compilers
  - web browsers, gaming engines
  - command shells and console utilities
  - Internet of Things (IoT) firmwares
- Why?
  - Language adds minimal run time overheads which means faster performance
  - Raw memory pointers provide more flexibility

Note: "Systems software" is in contrast with "application software".

Function calls and stack frames

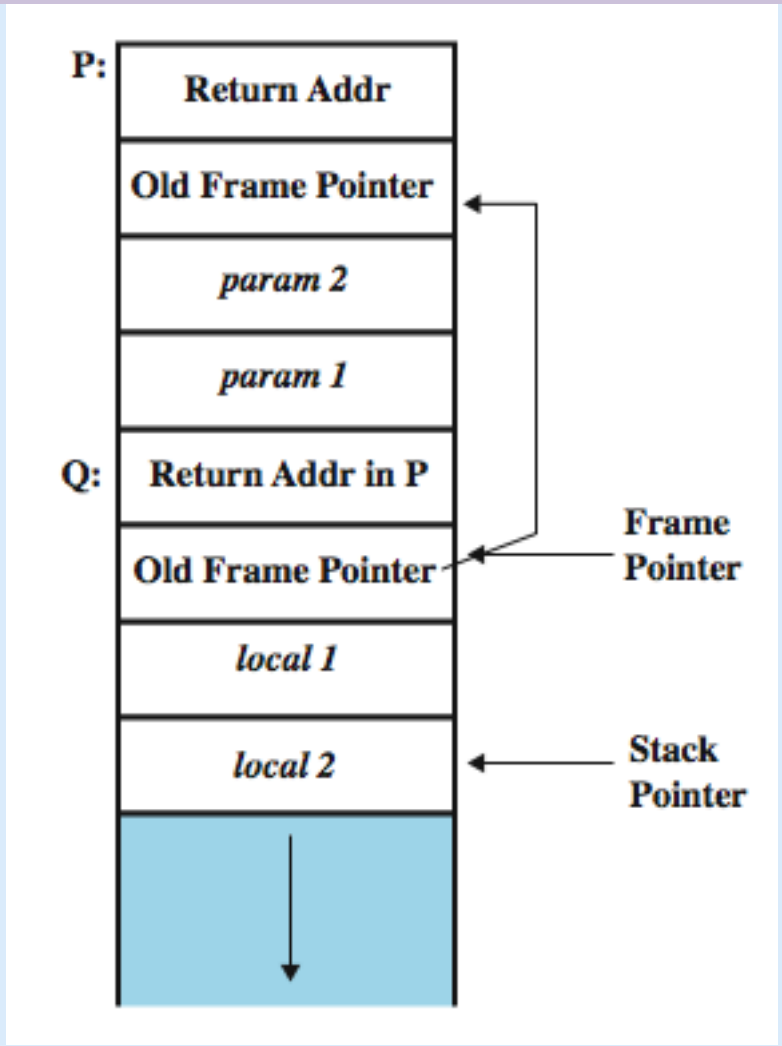


Stack frame

Calling function: needs a data structure to store the "return" address and parameters to be passed

Called function: needs a place to store its local variables somewhere different for every call

P function calling Q function



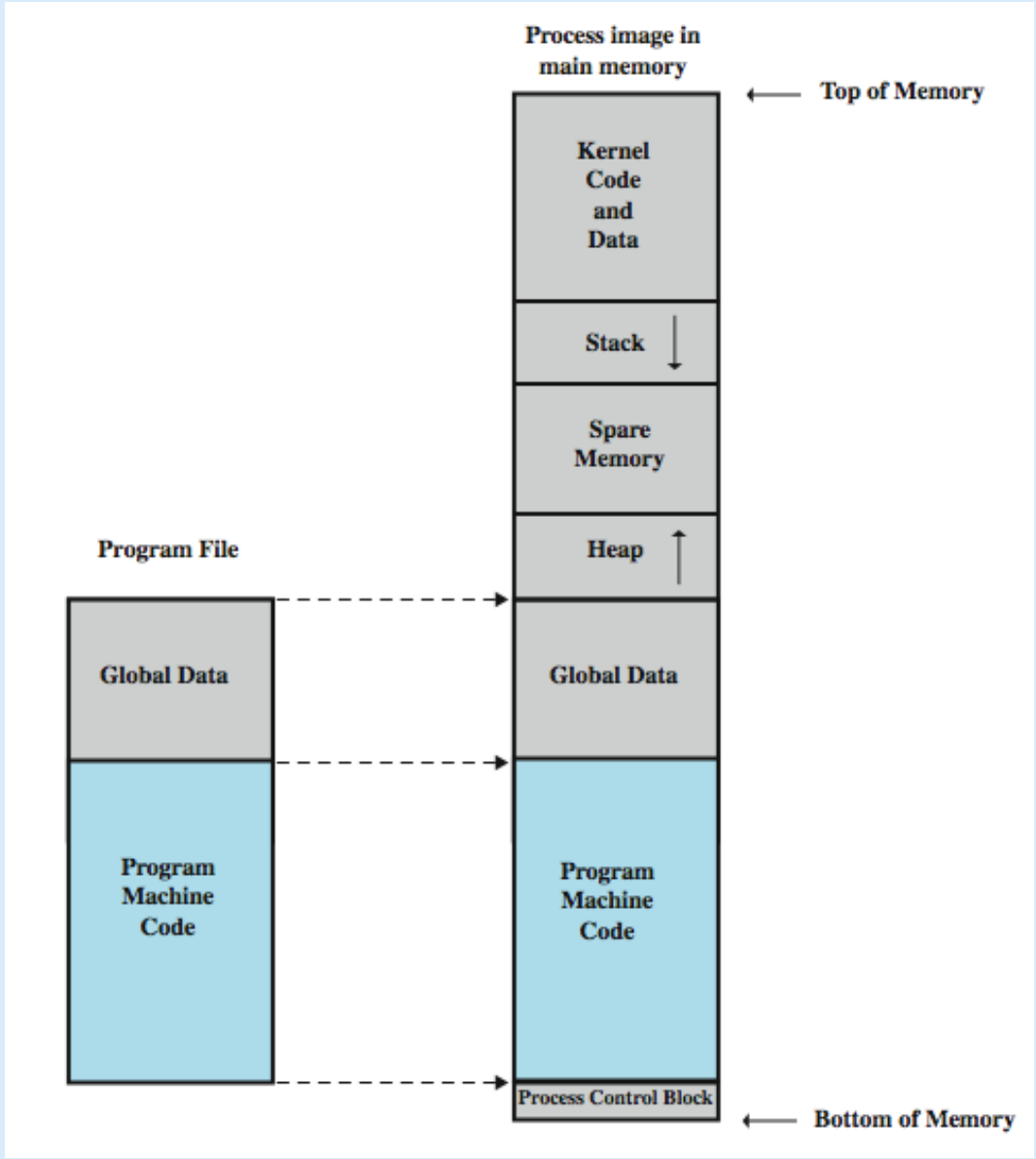
Stack Buffer Overflow



- Occurs when buffer is located on stack
  - used by Morris Worm
  - "Smashing the Stack" paper popularized it
- We have local variables below saved frame pointer and return address
  - hence overflow of a local buffer can potentially overwrite these key control items
- Attacker overwrites return address with address of a different code
  - like another function in same program that disables security
  - or a malicious code that has been inserted by attacker in heap, or in this buffer itself

Programs and Processes





Stack overflow example 2

```
void gctinp(ohar *inp, int siz)
{
    puts("Input value: ");
    fgets(inp, siz, stdin);
    printf("buffer3 getinp read %s\n", inp);
}

void display(char *val)
{
    char tmp[16];
    sprintf(tmp, "read val: %s\n", val);
    puts(tmp);
}

int main(int argc, char *argv[])
{
    char buf[16];
    getinp (buf, sizeof (buf));
    display(buf);
    printf("buffer3 done\n");
}
```

safe input function

sprintf is unsafe — when preparing a string longer than `tmp`, it will overwrite parts of the stack frame.



Stack overflow example 2

```
$ cc -o buffer3 buffer3.c

$ ./buffer3
Input value:
SAFE
buffer3 getinp read SAFE
read val: SAFE
buffer3 done

$ ./buffer3
Input value:
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
buffer3 getinp read XXXXXXXXXXXXXXX
read val: XXXXXXXXXXXXXXX

buffer3 done
Segmentation fault (core dumped)
```

Safe input function: reads only 15 characters

Crash because stack frame of `display()` was partially corrupted



Common unsafe C functions



Commonly used unsafe functions, and their safer alternatives that provide bounds checking

✗ `gets(char *str)` ✓ `fgets(char *str, int n, FILE *stream)`	Read line from standard input into `str`
✗ `strcat(char *dest, char *src)` ✓ `strncat(char *dest, const char *src, size_t n)`	Append contents of string `src` to string `dest`
✗ `strcpy(char *dest, char *src)` ✓ `snprintf` *or non standard* `strlcpy`	Copy contents of string `src` to `dest` overwriting previous
✗ `sprintf(char *str, char *format, ...)` ✓ `snprintf(char *str, size, char *format, ...)`	Create `str` according to supplied `format`
✗ `vsprintf(char *str, char *format, va_list arg)` ✓ `vsnprintf(char *str, size_t n, char* format, va_list arg)`	Create `str` according to supplied `format` and variables

Note that even the safe alternatives can result in buffer overflow if programmer passes an incorrectly large size in arguments!

Defenses



- Buffer overflows are widely exploited
- Large amount of vulnerable code is in use in real world projects
  - despite cause and countermeasures known

- Two broad defense approaches
  - compile-time
  - run-time
    - can't remove vulnerabilities, but make it harder for attacker to exploit them

Compile-Time Defenses



Choice of programming language (for new projects)

- Modern high level languages (Rust, Swift, Go etc.) apply strong typing and add additional code for bounds checking
  - not vulnerable to buffer overflow
  - compiler enforces permissible operations on variables
  - runtime checking crashes the programs for out-of-bounds read/write
    - crashing is not pretty, but safer than undefined behaviour of C
- These languages do have higher cost in resource use
- …and restrictions on access to hardware
  - so still need some code in C like languages

Compile-Time Defenses



Safe coding techniques

- If using potentially unsafe languages (like C), programmer must explicitly write safe code
  - by design with new code
  - **extensive code review** of existing code, (e.g. OpenBSD)

- Buffer overflow safety is a subset of general safe coding techniques

- Allow for graceful failure **(know how things may go wrong)**
  - check for sufficient space in any buffer

Compile-Time Defenses



Language extensions, Safe libraries

- Proposals for safety extensions to C — memory range checks added by compiler
  - performance penalties
  - must compile programs with special compiler

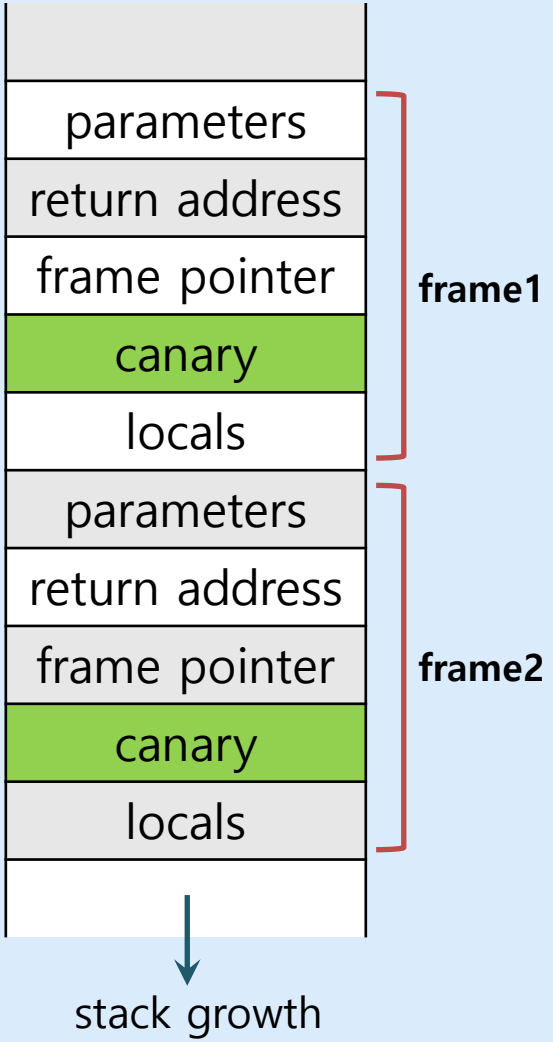
- Several safer standard library variants
  - new functions, e.g. strlcpy()
  - safer re-implementation of standard functions as a dynamic library, e.g. Libsafe

Compile-Time Defenses



Stack Protection

- Stackgaurd (GCC), Visual C++ /GS flag: add function entry and exit code to check stack for signs of corruption
  - place a randomly chosen value (canary) between local variables and saved frame pointer
  - upon function return, check if canary value has changed. Any buffer overflow attempt would overwrite this value first, before modifying frame pointer and return address
  - abort program if change found
  - issues: recompilation, debugger support



Compile-Time Defenses



Stack Protection

- Alternate method: at function entry, copy the return address in a safe, non-corruptible memory area
- upon function exit, check the return address on stack is the same as safe copy
  - abort the program in case changes are found
- format of stack remains unchanged.

- This approach is used in gcc extensions Stackshield and Return Address Defender (RAD).

Run-Time Defenses



Non Executable Address Space

aka Data Execution Prevention (DEP)

- Many BO attacks copy machine code into some buffer and transfer control to it
- Use virtual memory support to mark some regions of memory as data-only and non-executable (to avoid execution of attacker's code)
  - e.g. stack, heap, global data
  - need h/w support in Memory Management Unit (MMU)
- Issues: some applications need to put executable code in stack

Run-Time Defenses



Address Space Layout Randomization (ASLR)

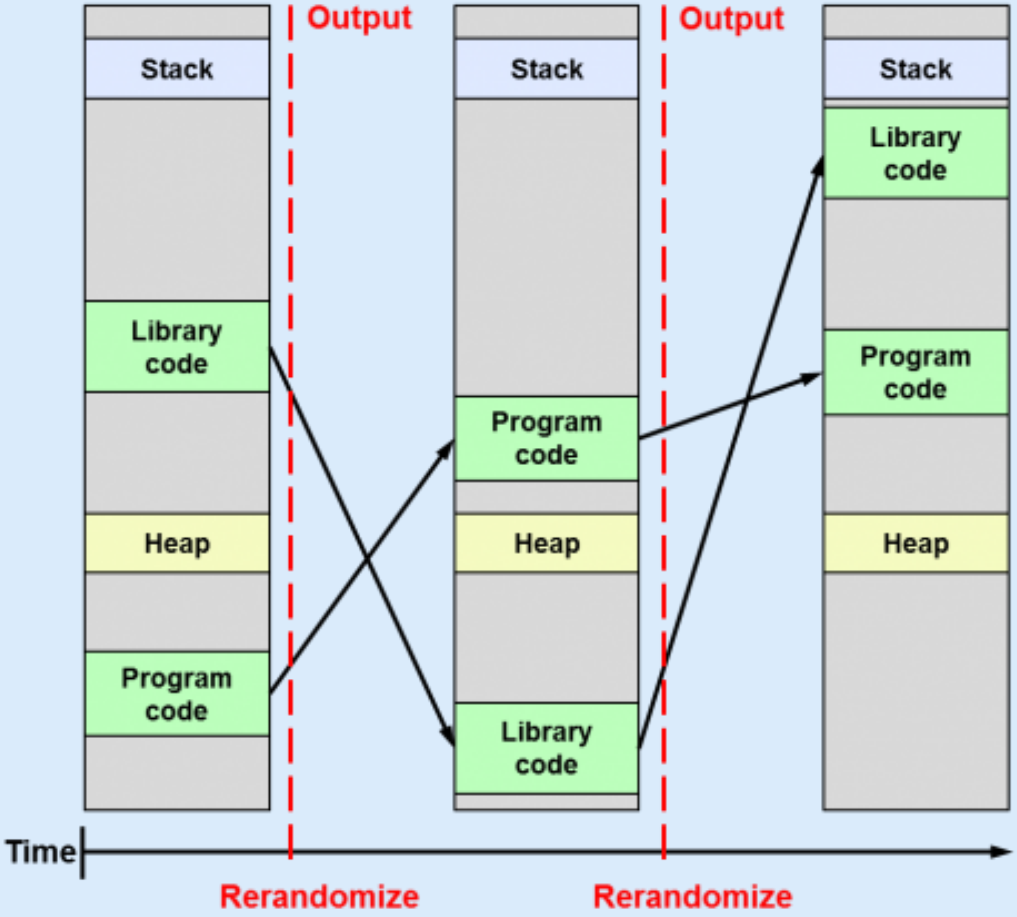
- Every time a process is created, randomize the location of key data structures in memory
  - stack, heap, global data: change addresses by 1 MB or so
  - using random shift for each process
  - modern systems have large address range, means wasting some has negligible impact

- Advantage: attacker can't predict the location of target buffer (containing the malicious code), so they can't transfer control to it

Run-Time Defenses



Address Space Layout Randomization (ASLR)

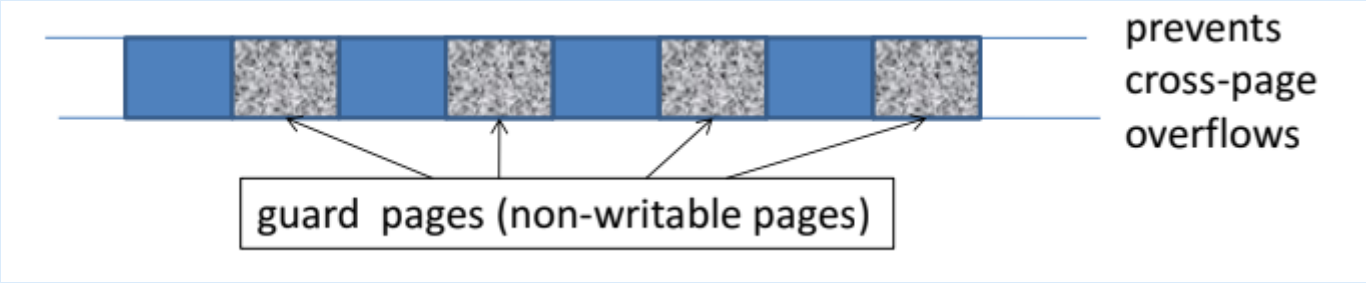


Run-Time Defenses



Guard Pages

- Place guard pages between critical regions of memory (or between stack frames)
  - flagged in MMU (mem management unit) as illegal addresses
  - any access aborts process



- Can even place between stack frames and heap buffers
  - at execution time and space cost

Other Overflow Attacks



- There is a range of other memory safety attack variants
  - stack overflow variants
  - heap overflow
  - global data overflow
  - format string overflow
  - integer overflow

- Most countermeasures mentioned earlier help in those attacks too.

References



- Stallings SPP chap 10
- Why are programs written in C and C++ so frequently vulnerable to overflow attacks: StackExchange Q/A